

1. Framework & Design Level

1. How do you design a scalable Playwright automation framework from scratch?

I would design the framework in layers so that tests contain business scenarios rather than low-level implementation details.

A typical structure:

```
playwright-framework/  
├── tests/  
│   ├── ui/  
│   ├── api/  
│   └── regression/  
├── pages/  
│   ├── LoginPage.ts  
│   ├── DashboardPage.ts  
│   └── CheckoutPage.ts  
├── fixtures/  
│   └── testFixtures.ts  
├── utils/  
│   ├── apiClient.ts  
│   ├── testData.ts  
│   └── logger.ts  
├── data/  
│   ├── users.json  
│   └── testData.json  
├── api/  
│   └── endpoints.ts  
├── playwright.config.ts  
├── package.json  
└── reports/
```

Key principles:

- Use Page Object Model for UI abstraction.
- Use fixtures for common setup and dependency injection.
- Keep test data separate from test logic.
- Create reusable API clients/utilities.
- Support multiple environments through configuration.
- Enable parallel execution.

- Capture traces/screenshots/videos on failures.
- Generate CI-friendly reports.
- Avoid hard waits.
- Keep tests independent and deterministic.

The most important goal is maintainability and isolation, not simply creating a large number of utility classes.

2. How do you implement Page Object Model in Playwright?

Each page gets a class containing locators and page-specific actions.

```
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly username: Locator;
  readonly password: Locator;
  readonly loginButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.username = page.getByLabel('Username');
    this.password = page.getByLabel('Password');
    this.loginButton = page.getByRole('button', { name: 'Login'
});
  }

  async login(username: string, password: string) {
    await this.username.fill(username);
    await this.password.fill(password);
    await this.loginButton.click();
  }
}
```

Test:

```
test('valid login', async ({ page }) => {
  const loginPage = new LoginPage(page);

  await loginPage.login('admin', 'password');

  await expect(page).toHaveURL(/dashboard/);
});
```

```
});
```

I prefer exposing business-level methods such as `login()`, `createOrder()` and `checkout()` rather than exposing every click and locator to the test.

3. How do you manage test data in Playwright?

I use different mechanisms depending on the type of data:

- JSON — static test data.
- CSV — data-driven scenarios.
- Environment variables — credentials/configuration.
- API — dynamically create test data.
- Database — when application architecture allows controlled DB access.
- Fixtures — reusable generated data.

For example:

```
import users from '../data/users.json';

test('login', async ({ page }) => {
  await loginPage.login(
    users.validUser.username,
    users.validUser.password
  );
});
```

For large suites, I prefer API-generated test data because it reduces UI setup time and makes tests more independent.

4. How do you implement environment-based execution?

I usually use environment variables.

```
ENV=qa npx playwright test
```

Configuration:

```
const environments = {
  dev: {
    baseURL: 'https://dev.example.com',
  },
  qa: {
    baseURL: 'https://qa.example.com',
  },
};
```

```

    prod: {
      baseURL: 'https://example.com',
    }
  };

const env = process.env.ENV || 'qa';

export default defineConfig({
  use: {
    baseURL: environments[env].baseURL
  }
});

```

For credentials, I never hardcode passwords. I use CI secrets or environment variables.

5. How do you handle multi-environment configurations in playwright.config.ts?

I keep environment-specific values in a configuration object or external configuration files.

```

const config = {
  dev: {
    baseURL: 'https://dev.example.com',
    apiURL: 'https://dev-api.example.com'
  },
  qa: {
    baseURL: 'https://qa.example.com',
    apiURL: 'https://qa-api.example.com'
  },
  prod: {
    baseURL: 'https://prod.example.com',
    apiURL: 'https://api.example.com'
  }
};

const environment = process.env.ENV ?? 'qa';

export default defineConfig({
  use: {
    baseURL: config[environment].baseURL
  }
});

```

```
}  
});
```

I would also validate that required environment variables exist so that a CI job doesn't accidentally run against the wrong environment.

6. How do you structure large test suites?

I organize tests based on business domains, rather than putting hundreds of tests into one directory.

```
tests/  
├── authentication/  
├── customers/  
├── orders/  
├── payments/  
├── reports/  
└── regression/
```

Common principles:

- Tests should be independent.
- Reusable functionality goes into pages/services/helpers.
- Common setup goes into fixtures.
- Test data stays outside test logic.
- Smoke, regression and API tests can be tagged.
- Avoid deeply nested inheritance.
- Avoid putting assertions exclusively inside page objects.

Example:

```
npx playwright test --grep @smoke  
npx playwright test --grep @regression
```

7. How do you implement custom utilities/helpers?

I create utilities only for genuinely reusable functionality.

```
export async function waitForApiResponse(  
  page: Page,  
  url: string  
) {  
  return page.waitForResponse(response =>  
    response.url().includes(url) &&
```

```
        response.ok()
    );
}
```

I avoid creating generic helpers like:

```
clickElement()
enterText()
wait()
```

for every simple Playwright operation because that can hide Playwright's built-in functionality and make debugging harder.

2. API + UI Integration

1. How do you perform API testing in Playwright?

Playwright provides request.

```
import { test, expect } from '@playwright/test';

test('GET customer', async ({ request }) => {
    const response = await request.get('/api/customers/123');

    expect(response.ok()).toBeTruthy();
    expect(response.status()).toBe(200);

    const body = await response.json();

    expect(body.id).toBe(123);
});
```

For POST:

```
const response = await request.post('/api/customers', {
    data: {
        name: 'John',
        email: 'john@test.com'
    }
});

expect(response.status()).toBe(201);
```

2. How do you reuse API responses in UI tests?

A common approach is to create data through API and then validate it through UI.

```
const response = await request.post('/api/orders', {
  data: {
    productId: 100,
    quantity: 2
  }
});
```

```
const order = await response.json();
```

```
await page.goto('/orders');
```

```
await expect(
  page.getByText(order.orderNumber)
).toBeVisible();
```

This is much faster than creating an order entirely through the UI.

3. How do you handle authentication using API?

I can authenticate through an API and save the resulting session.

```
const response = await request.post('/api/login', {
  data: {
    username: process.env.USERNAME,
    password: process.env.PASSWORD
  }
});
```

```
const data = await response.json();
```

```
const token = data.token;
```

Then use the token in subsequent API requests:

```
await request.get('/api/orders', {
  headers: {
    Authorization: `Bearer ${token}`
  }
});
```

For browser authentication, I generally prefer generating a valid authenticated `storageState` when possible.

4. How do you mock API responses using `route.fulfill()`?

```
await page.route('**/api/products', async route => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({
      products: [
        {
          id: 1,
          name: 'Mock Product',
          price: 100
        }
      ]
    })
  });
});

await page.goto('/products');
```

This is useful for testing:

- Error responses.
 - Empty states.
 - Slow APIs.
 - Rare backend conditions.
 - UI behavior independently of backend availability.
-

5. Difference between `APIRequestContext` and `Page` request?

`APIRequestContext` is designed for direct HTTP/API operations.

```
const response = await request.get('/api/users');
```

The page object is primarily for browser/UI automation.

There is also `page.request`, which provides an API request context associated with the browser context.

The important distinction is that API requests don't require browser rendering, DOM interaction or navigation.

3. Advanced Locators & Selectors

1. Difference between locator() and page.\$()?

locator() is the preferred modern approach.

```
const button = page.getByRole('button', { name: 'Submit' });
await button.click();
```

page.\$() returns an ElementHandle.

```
const button = await page.$('#submit');
```

Locator provides:

- Auto-waiting.
- Retryability.
- Better handling of dynamic DOM.
- Better assertions.
- Lazy evaluation.

Therefore, I prefer Locator APIs over ElementHandles for normal UI automation.

2. What is auto-waiting?

Playwright automatically waits for elements to become actionable before performing actions.

For example:

```
await page.getByRole('button', { name: 'Submit' }).click();
```

Playwright checks conditions such as:

- Element exists.
- Element is visible.
- Element is enabled.
- Element is stable.
- Element can receive the click.

This significantly reduces explicit synchronization code.

3. How do you handle dynamic elements?

I avoid brittle selectors such as:

```
div:nth-child(7)
```

Instead I use semantic locators:

```
page.getByRole('button', { name: 'Submit' });
page.getByLabel('Email');
page.getByPlaceholder('Enter email');
page.getByTestId('submit-button');
```

For dynamic text:

```
page.getByText(/Order #\d+/);
```

For collections:

```
const rows = page.locator('table tbody tr');

await expect(rows).toHaveCount(5);
```

4. How do you handle Shadow DOM?

Playwright locators generally work through open Shadow DOM automatically.

```
await page.locator('my-component').getByRole('button').click();
```

You can also chain locators:

```
const component = page.locator('my-component');

await component.getByText('Submit').click();
```

Closed Shadow DOM cannot generally be accessed through normal DOM selectors.

5. How do you create custom locator strategies?

I first prefer built-in semantic locators.

If the application consistently provides a custom attribute:

```
<button data-testid="checkout-button">
```

I use:

```
page.getByTestId('checkout-button');
```

I would configure a custom test ID attribute if the application uses something else:

```
use: {  
  testIdAttribute: 'data-qa'  
}
```

Then:

```
page.getByTestId('checkout-button');
```

4. Synchronization

1. Difference between `waitForSelector()`, `waitForLoadState()`, `waitForTimeout()`?

`waitForSelector()`

Waits for a particular DOM element/state.

```
await page.locator('#result').waitFor({  
  state: 'visible'  
});
```

`waitForLoadState()`

Waits for a page loading state.

```
await page.waitForLoadState('networkidle');
```

Although I don't use `networkidle` blindly because modern applications may continuously make network requests.

`waitForTimeout()`

A fixed delay:

```
await page.waitForTimeout(3000);
```

I avoid this in normal automation because it creates unnecessary delays and can still be flaky.

2. Why is hard wait discouraged?

Consider:

```
await page.waitForTimeout(5000);
```

If the application is ready after 500 ms, we waste 4.5 seconds.

If the application needs 6 seconds, the test still fails.

Instead, wait for a meaningful condition:

```
await expect(page.getByText('Order created')).toBeVisible();
```

or:

```
await page.waitForResponse('**/api/orders');
```

3. How does Playwright handle implicit waits?

Playwright performs auto-waiting for actions and assertions.

For example:

```
await page.getByRole('button', { name: 'Save' }).click();
```

Playwright waits until the button is actionable.

Assertions also retry:

```
await expect(
  page.getByText('Success')
).toBeVisible();
```

The assertion retries until the configured timeout expires.

5. Parallel Execution & Performance

1. How does Playwright achieve parallel execution?

Playwright Test uses worker processes.

Tests can run concurrently across workers:

```
npx playwright test --workers=4
```

Each worker gets its own isolated test environment according to Playwright's test isolation model.

This allows multiple tests to execute simultaneously.

2. Difference between Workers and Projects?

Workers

Control parallel execution capacity.

```
--workers=4
```

Projects

Represent different configurations.

For example:

```
projects: [  
  {  
    name: 'chromium',  
    use: { browserName: 'chromium' }  
  },  
  {  
    name: 'firefox',  
    use: { browserName: 'firefox' }  
  }  
]
```

A project can represent:

- Browser.
- Device.
- Environment.
- Configuration variation.

3. How do you control test execution threads?

In Playwright terminology, I configure workers.

CLI:

```
npx playwright test --workers=4
```

Config:

```
workers: process.env.CI ? 2 : 4
```

For CI, I choose the worker count based on available CPU/memory and test behavior.

4. How do you run tests in sharding mode?

For example:

```
npx playwright test --shard=1/4
```

Then four CI jobs can run:

```
--shard=1/4  
--shard=2/4  
--shard=3/4  
--shard=4/4
```

This distributes the test suite across CI machines.

5. How do you reduce test execution time?

I would:

- Run independent tests in parallel.
 - Use API setup instead of UI setup.
 - Reuse authenticated storageState.
 - Avoid waitForTimeout.
 - Avoid unnecessary navigation.
 - Block unnecessary resources where appropriate.
 - Use sharding in CI.
 - Keep smoke tests separate from full regression.
 - Optimize test data creation.
 - Avoid testing the same workflow repeatedly through the UI.
-

6. Network Handling

1. How do you intercept network requests?

```
await page.route('**/api/users', async route => {  
  console.log(route.request().method());  
});
```

```
    await route.continue();
  });
```

You can also modify requests:

```
await page.route('**/api/users', async route => {
  const headers = {
    ...route.request().headers(),
    'x-test': 'true'
  };

  await route.continue({ headers });
});
```

2. How do you validate an API response in UI tests?

```
const responsePromise = page.waitForResponse(
  response =>
    response.url().includes('/api/orders') &&
    response.request().method() === 'POST'
);

await page.getByRole('button', { name: 'Place Order' }).click();

const response = await responsePromise;

expect(response.status()).toBe(201);

const body = await response.json();

expect(body.status).toBe('created');
```

3. How do you simulate network failure or slow network?

Failure:

```
await page.route('**/api/products', async route => {
  await route.abort();
});
```

Slow response:

```
await page.route('**/api/products', async route => {
  await new Promise(resolve => setTimeout(resolve, 5000));
  await route.continue();
});
```

Then verify the UI displays the correct loading/error state.

4. How do you block specific requests?

For example, block images:

```
await page.route('**/*', async route => {
  if (route.request().resourceType() === 'image') {
    await route.abort();
  } else {
    await route.continue();
  }
});
```

This can improve performance in certain test scenarios, although I wouldn't do it globally if image loading itself is under test.

7. Authentication & Session

1. How do you handle login once and reuse the session?

I use storageState.

A setup test logs in once:

```
await page.goto('/login');

await page.getByLabel('Username').fill('admin');
await page.getByLabel('Password').fill('password');
await page.getByRole('button', { name: 'Login' }).click();

await page.context().storageState({
  path: 'playwright/.auth/admin.json'
});
```

Then configure:

```
use: {
```

```
storageState: 'playwright/.auth/admin.json'
}
```

This avoids logging in through the UI for every test.

2. What is storageState?

storageState stores browser authentication state such as:

- Cookies.
- Local storage.

It allows a new browser context to start already authenticated.

Example:

```
storageState: 'playwright/.auth/user.json'
```

It is particularly useful for large test suites.

The generated authentication file should generally be treated as sensitive and excluded from source control.

3. How do you implement SSO login?

For SSO, I first determine whether the identity provider supports a stable non-interactive authentication mechanism for automation.

Possible approaches:

- Pre-authenticated storageState.
- API-based authentication.
- Dedicated test identity provider/environment.
- OAuth/token-based setup.
- Browser-based SSO flow when necessary.

I avoid automating real MFA/SSO infrastructure in every test if a secure test authentication mechanism is available.

4. How do you handle multi-user scenarios?

I create separate authentication states.

```
.auth/  
├─ admin.json
```

```
|— customer.json
|— manager.json
```

Then configure separate projects or fixtures.

Example:

```
test.use({
  storageState: 'playwright/.auth/admin.json'
});
```

For two users interacting simultaneously, I can create two separate browser contexts:

```
const adminContext = await browser.newContext({
  storageState: 'playwright/.auth/admin.json'
});
```

```
const userContext = await browser.newContext({
  storageState: 'playwright/.auth/user.json'
});
```

8. CI/CD

1. How do you integrate Playwright with Jenkins, GitHub Actions or Azure DevOps?

The basic pipeline is:

Checkout code

↓

Install dependencies

↓

Install Playwright browsers

↓

Set environment variables/secrets

↓

Run tests

↓

Publish report

↓

Upload artifacts

Typical command:

```
npm ci
npx playwright install --with-deps
npx playwright test
```

In CI, I configure retries and artifacts appropriately.

2. How do you generate reports in CI?

For example:

```
reporter: [
  ['html'],
  ['junit', { outputFile: 'results.xml' }]
]
```

HTML is useful for human investigation.

JUnit is useful for CI systems that consume XML test results.

3. How do you store artifacts?

I configure:

```
use: {
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
  trace: 'retain-on-failure'
}
```

Then CI uploads:

```
playwright-report/
test-results/
screenshots/
videos/
traces/
```

4. How do you run Playwright in Docker?

I can use the official Playwright Docker image or build an image containing the required Node.js and browser dependencies.

A typical Dockerfile conceptually looks like:

```
FROM mcr.microsoft.com/playwright:<version>
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm ci
```

```
COPY . .
```

```
CMD ["npx", "playwright", "test"]
```

I pin the Playwright version so the browser binaries and test dependencies remain consistent.

9. Debugging & Stability

1. How do you debug flaky tests?

My approach is:

1. Check the trace.
2. Check screenshots/video.
3. Identify whether failure is timing, locator, data or environment related.
4. Look at network/API failures.
5. Run the test repeatedly.
6. Run it under CI-like conditions.
7. Remove unnecessary waits and replace them with condition-based waits.
8. Verify test isolation.
9. Check whether another test modifies shared state.

I don't simply add retries and consider the problem solved.

2. What is Trace Viewer?

Trace Viewer records detailed test execution information.

It can contain:

- Actions.
- DOM snapshots.
- Screenshots.

- Network activity.
- Console information.
- Timing information.

Run:

```
npx playwright show-trace trace.zip
```

This is one of my first tools when investigating CI-only failures.

3. How do you capture screenshots/videos on failure?

Configuration:

```
use: {  
  screenshot: 'only-on-failure',  
  video: 'retain-on-failure',  
  trace: 'retain-on-failure'  
}
```

This keeps artifacts manageable while preserving evidence for failed tests.

4. What is --debug mode?

```
npx playwright test --debug
```

It launches tests in a debugging-friendly mode and allows inspection of actions and locators.

I can also use:

```
npx playwright test tests/login.spec.ts --debug
```

For deeper investigation, I use the Playwright Inspector and trace viewer.

10. Cross-Browser & Mobile

1. How do you run tests on Chromium, Firefox and WebKit?

Configure projects:

```
projects: [  
  {  
    name: 'chromium',  
    use: { browserName: 'chromium' }  
  }  
]
```

```
    },  
    {  
      name: 'firefox',  
      use: { browserName: 'firefox' }  
    },  
    {  
      name: 'webkit',  
      use: { browserName: 'webkit' }  
    }  
  ]  
}
```

Run all:

```
npx playwright test
```

Only Chromium:

```
npx playwright test --project=chromium
```

2. How do you emulate mobile devices?

Playwright provides device descriptors.

```
import { devices } from '@playwright/test';
```

```
projects: [  
  {  
    name: 'Mobile Chrome',  
    use: {  
      ...devices['Pixel 5']  
    }  
  }  
]
```

3. What is device emulation?

It simulates characteristics of a device, such as:

- Screen size.
- User agent.
- Device scale factor.
- Touch support.
- Mobile viewport behavior.

It is not identical to testing on a real physical device, so for critical mobile behavior I would complement it with real-device coverage where required.

11. File Handling

1. How do you handle file upload?

```
await page
  .getByLabel('Upload file')
  .setInputFiles('test-data/sample.pdf');
```

For a file chooser:

```
const fileChooserPromise = page.waitForEvent('filechooser');

await page.getByRole('button', { name: 'Upload' }).click();

const fileChooser = await fileChooserPromise;

await fileChooser.setFiles('test-data/sample.pdf');
```

2. How do you handle file download and validate content?

```
const downloadPromise = page.waitForEvent('download');

await page.getByText('Download').click();

const download = await downloadPromise;

const path = await download.path();

console.log(path);
```

I can save it:

```
await download.saveAs('test-results/report.pdf');
```

For CSV/text files, I can read and validate their contents:

```
import fs from 'fs';

const content = fs.readFileSync(
  'test-results/report.csv',
```

```
    'utf-8'  
  );  
  
expect(content).toContain('Order ID');
```

12. Advanced Scenarios

1. How do you handle iframe?

Use `frameLocator()`:

```
const frame = page.frameLocator('#payment-frame');  
  
await frame.getByLabel('Card Number').fill('4111111111111111');
```

For a frame object:

```
const frame = page.frame({ name: 'payment-frame' });
```

I prefer `frameLocator()` for locator-based interactions.

2. How do you handle multiple tabs/windows?

Use `context.waitForEvent('page')`:

```
const newPagePromise = page.context().waitForEvent('page');  
  
await page.getByText('Open Report').click();  
  
const newPage = await newPagePromise;  
  
await newPage.waitForLoadState();  
  
await expect(newPage).toHaveTitle(/Report/);
```

For a popup:

```
const popupPromise = page.waitForEvent('popup');  
  
await page.getByText('Open').click();  
  
const popup = await popupPromise;
```

3. How do you handle alerts/popups?

Playwright automatically handles dialogs only when there is no dialog listener; I generally explicitly handle expected dialogs.

```
page.on('dialog', async dialog => {  
  console.log(dialog.message());  
  await dialog.accept();  
});
```

For confirmation:

```
page.once('dialog', async dialog => {  
  await dialog.dismiss();  
});
```

4. How do you test drag and drop?

If supported by the application:

```
await page  
  .locator('#source')  
  .dragTo(page.locator('#target'));
```

For more complex custom drag-and-drop implementations, I may use mouse actions:

```
await page.locator('#source').hover();  
await page.mouse.down();
```

```
await page.locator('#target').hover();  
await page.mouse.up();
```

5. How do you handle infinite scrolling?

I scroll until the required data appears or no new content is loaded.

```
while (!(await page.getByText('Target Item')).isVisible())) {  
  await page.mouse.wheel(0, 1000);  
  await page.waitForTimeout(500);  
}
```

For a production framework, I'd prefer waiting on a meaningful loading indicator/API response rather than using a fixed timeout.

For example, wait for the next API response and continue scrolling until the target appears.

6. How do you validate table data dynamically?

```
const rows = page.locator('table tbody tr');

const count = await rows.count();

for (let i = 0; i < count; i++) {
  const row = rows.nth(i);

  const text = await row.innerText();

  console.log(text);
}
```

For a specific condition:

```
const row = page
  .locator('table tbody tr')
  .filter({ hasText: 'John' });

await expect(row).toContainText('Active');
```

I prefer locator filtering rather than relying on hard-coded row indexes.

13. Real-Time Scenarios

1. Test is failing randomly — how will you debug?

I'd follow a systematic process:

```
Reproduce
  ↓
Check trace
  ↓
Check screenshot/video
  ↓
Check network/API
  ↓
Check locator stability
```

↓
Check synchronization
↓
Check test data
↓
Check test isolation
↓
Run under CI conditions

I would ask:

- Is the locator unstable?
- Is the backend response delayed?
- Is data shared between tests?
- Is there a race condition?
- Is the test dependent on execution order?
- Is the environment overloaded?

Only after identifying the cause would I consider increasing retries.

2. How will you automate OTP-based login?

For a test environment, I prefer a controlled solution such as:

- Test-only OTP bypass.
- API endpoint that generates/retrieves a test OTP.
- Dedicated test OTP service.
- Database/API retrieval if explicitly supported by the test environment.

I would not try to bypass production security mechanisms.

For example, if the test environment exposes:

```
POST /test/otp
```

I could retrieve the OTP using Playwright's API request and enter it into the UI.

3. How will you test CAPTCHA-protected pages?

I don't attempt to solve or bypass a real CAPTCHA programmatically.

For automation environments, I would request:

- CAPTCHA disabled in the test environment.
- Official test mode provided by the CAPTCHA vendor.

- A controlled test bypass.
- Dedicated non-CAPTCHA authentication path.

The objective is to test the application's business flow without undermining the security control.

4. How will you test a payment flow?

For payment systems, I use the payment provider's sandbox/test environment.

Typical approach:

```
Create test customer/order via API
      ↓
Open checkout UI
      ↓
Use provider's test card/data
      ↓
Submit payment
      ↓
Wait for payment API response
      ↓
Validate UI confirmation
      ↓
Validate backend/payment status
```

I also test:

- Successful payment.
- Declined payment.
- Expired card.
- 3DS/test authentication scenarios.
- Timeout.
- Duplicate submission.
- Payment cancellation.

I never use real payment credentials or real transactions in automation.

5. How do you handle flaky elements in CI but not locally?

First, I compare:

- Browser/version.
- CPU/memory.

- Network.
- Environment.
- Timing.
- Test order.
- Parallelism.

Then inspect the trace.

Common causes are:

- Race conditions.
- Weak locators.
- Backend delays.
- Shared test data.
- Resource contention.
- Missing synchronization.

I replace timing assumptions with condition-based synchronization.

For example, instead of:

```
await page.waitForTimeout(3000);
await button.click();
```

use:

```
await expect(button).toBeEnabled();
await button.click();
```

6. How do you implement retry mechanism?

In configuration:

```
export default defineConfig({
  retries: process.env.CI ? 2 : 0
});
```

I use retries mainly as a safety net, not as a solution for broken tests.

A useful strategy is:

```
Local: 0 retries
CI:    1-2 retries
```

If a test passes only after retries repeatedly, I treat it as a flaky test requiring investigation.

14. Coding-Based Questions

1. Write code to login and validate dashboard

```
import { test, expect } from '@playwright/test';

test('login and validate dashboard', async ({ page }) => {
  await page.goto('/login');

  await page.getByLabel('Username').fill(
    process.env.USERNAME!
  );

  await page.getByLabel('Password').fill(
    process.env.PASSWORD!
  );

  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page).toHaveURL(/dashboard/);

  await expect(
    page.getByRole('heading', { name: 'Dashboard' })
  ).toBeVisible();
});
```

2. Wait for API response

```
const responsePromise = page.waitForResponse(
  response =>
    response.url().includes('/api/orders') &&
    response.request().method() === 'GET' &&
    response.status() === 200
);

await page.getByRole('button', { name: 'Orders' }).click();

const response = await responsePromise;
```

```
const body = await response.json();

expect(body.orders).toBeDefined();
```

The important point is that I create the `waitForResponse()` promise before triggering the action.

3. Upload file

```
await page
  .getByLabel('Choose file')
  .setInputFiles('test-data/sample.pdf');

await expect(
  page.getByText('sample.pdf')
).toBeVisible();
```

4. Handle dropdown

For a native `<select>`:

```
await page
  .getByLabel('Country')
  .selectOption('IN');
```

For a custom dropdown:

```
await page.getByRole('combobox', { name: 'Country' }).click();

await page.getByRole('option', {
  name: 'India'
}).click();
```

The implementation depends on whether the application uses a native select or a custom component.

5. Write a reusable function for common actions

A good reusable function should represent a meaningful operation.

For example:

```

async function login(
  page: Page,
  username: string,
  password: string
) {
  await page.goto('/login');

  await page.getByLabel('Username').fill(username);
  await page.getByLabel('Password').fill(password);

  await page.getByRole('button', {
    name: 'Login'
  }).click();

  await expect(
    page.getByRole('heading', { name: 'Dashboard' })
  ).toBeVisible();
}

```

Then:

```

test('admin login', async ({ page }) => {
  await login(
    page,
    process.env.ADMIN_USER!,
    process.env.ADMIN_PASSWORD!
  );
});

```

15. High-Value Interview Follow-Up Questions

For a 4+ years Playwright role, interviewers often go beyond definitions and ask architectural follow-ups.

Be prepared for questions such as:

Why Playwright over Selenium?

A strong answer:

Playwright provides modern browser automation with built-in auto-waiting, browser-context isolation, network interception, API testing, tracing, multi-browser support and strong parallel execution. It also provides first-class support for Chromium, Firefox and WebKit from one framework.

Why POM?

POM separates test intent from UI implementation. If a locator changes, I update the page object instead of changing dozens of tests. However, I avoid overengineering POM and keep business workflows and assertions at the appropriate layer.

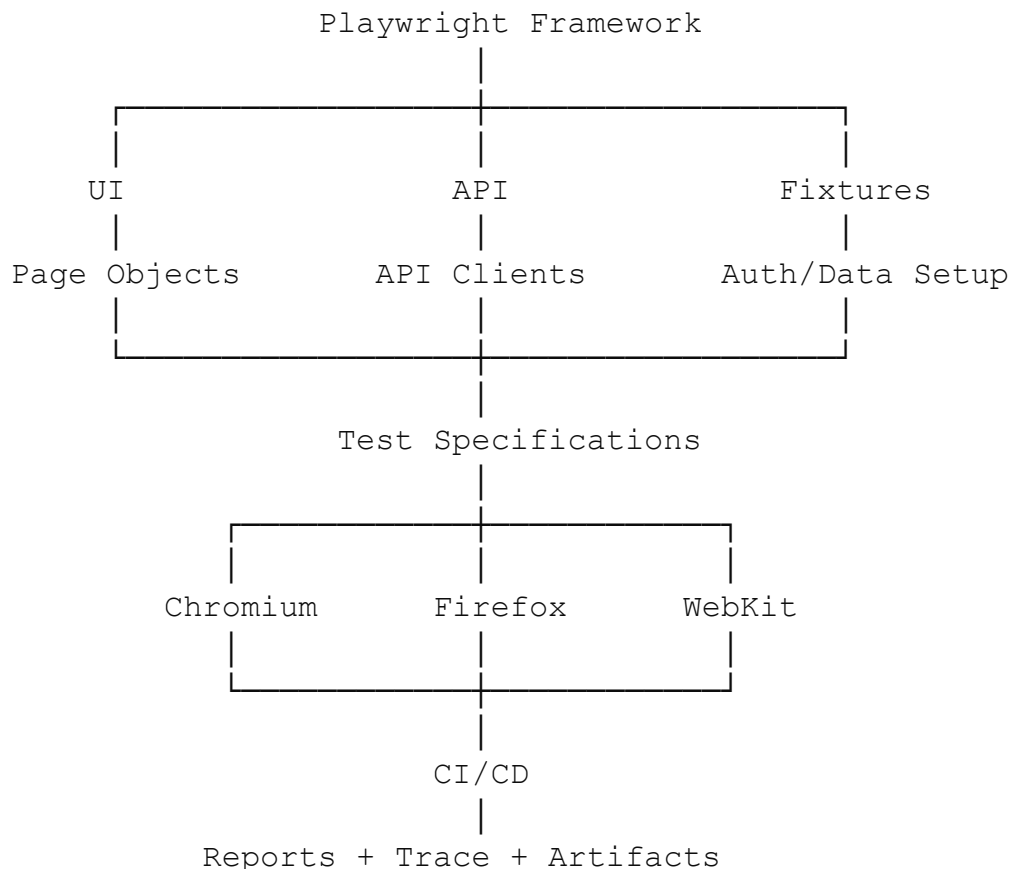
Why API + UI combination?

UI tests are expensive and slower. I use APIs to prepare data and authentication, then use the UI only to validate user-facing behavior. This gives better speed and test isolation.

How do you make automation reliable?

Reliable automation comes from deterministic test data, independent tests, stable locators, condition-based synchronization, isolated browser contexts, controlled environments, API-based setup, meaningful assertions and good failure diagnostics. Retries should be a safety net, not the primary stability mechanism.

What would your ideal framework contain?



16. A concise interview strategy

For a 4+ year interview, don't answer only with definitions. A strong pattern is:

Concept → Why → Implementation → Real project example → Trade-off

For example, for storageState:

"storageState stores browser authentication state such as cookies and local storage. I use it to avoid repeating UI login in every test. In my framework, a setup project authenticates the user once and generates an auth state file. Tests consume that state through use.storageState. For multi-user scenarios, I maintain separate states or create separate browser contexts. The main consideration is that authentication state is sensitive, so it should not be committed to source control."

That style demonstrates hands-on framework experience, rather than simply knowing Playwright syntax.